

Planejamento de Implementação vs. Desenvolvimento: Um Modelo Mental ao Qual Sempre Volto

Uma reflexão prática sobre como o planejamento de implementação pode reduzir esforço de desenvolvimento, sem fingir que o gráfico é um modelo científico.

Publicado: 9 de junho de 2026 · Atualizado: 5 de setembro de 2026

Artigo original: <https://www.cleisson.com/pt-BR/blog/implementation-planning-vs-developing>

No projeto em que atuo, passamos a criar tarefas separadas para planos de implementação. Em alguns casos, a tarefa de desenvolvimento só era criada, refinada ou dividida depois que a tarefa de planejamento terminava.

No começo, eu queria ir direto para o código. É ali que as coisas parecem concretas. Mas, depois de acompanhar esse processo algumas vezes, passei a valorizar o tempo para entender o que uma mudança afetaria antes de escolher uma abordagem. Em funcionalidades, refatorações, integrações e migrações, chegar a esse entendimento pode dar mais trabalho do que o ticket sugere.

Minha regra é planejar enquanto essa for a forma menos custosa de responder a uma pergunta capaz de mudar a implementação. Quando um experimento pequeno e reversível puder ensinar mais, é hora de construir. Avalio o plano pelo esforço e pelo risco que ele poupa ao longo da mudança inteira, incluindo o tempo gasto planejando.

Por planejamento de implementação, entendo ler o código existente, compreender as fronteiras do sistema, checar premissas e definir a estrutura, os testes e os passos manuais de que a mudança precisa. Desenvolvimento inclui escrever código, ajustar testes, revisar, integrar, depurar, fazer deploy e lidar com o que descobrimos pelo caminho.

Essas atividades se sobrepõem. Um pequeno spike de implementação pode fazer parte do planejamento, e o código funcionando pode mostrar que o plano estava errado. Tarefas separadas ajudam a organizar o trabalho; elas não fazem o aprendizado acontecer em linha reta.

Por que uma tarefa separada de planejamento pode ajudar

Sem uma etapa explícita de planejamento, a investigação costuma se espalhar por comentários em tickets, conversas no Slack e descobertas durante o desenvolvimento. Para uma mudança pequena, isso pode bastar. Para uma maior, quero que essas descobertas estejam em um lugar onde o time consiga usá-las antes de o trabalho depender de uma premissa não verificada.

Uma tarefa dedicada torna essa investigação visível. Dá ao engenheiro tempo para ler o código, perguntar quem é responsável por um fluxo e verificar se um padrão existente resolve parte do problema. O resultado pode ser uma nota curta com as áreas afetadas, uma sequência de implementação e os riscos que o time precisa discutir.

A tarefa de desenvolvimento que vem depois pode ter limites e expectativas de teste mais claros. Também pode incluir aqueles passos manuais chatos que são fáceis de esquecer até o dia do release.

Eu ajustaria o plano às consequências de errar na mudança. Uma alteração local e fácil de desfazer talvez precise apenas de uma nota no ticket. Uma migração com dependências e comportamentos legados merece uma investigação mais cuidadosa. Uma tarefa separada de planejamento é útil quando abre espaço para esse trabalho; eu não a tornaria obrigatória para toda alteração de código.

O que quero que um plano descubra

Um ticket raramente contém toda a história de um sistema. Regras de negócio, concessões antigas e áreas marcadas como *"não mexa nisso a menos que você saiba o motivo"* podem mudar a abordagem.

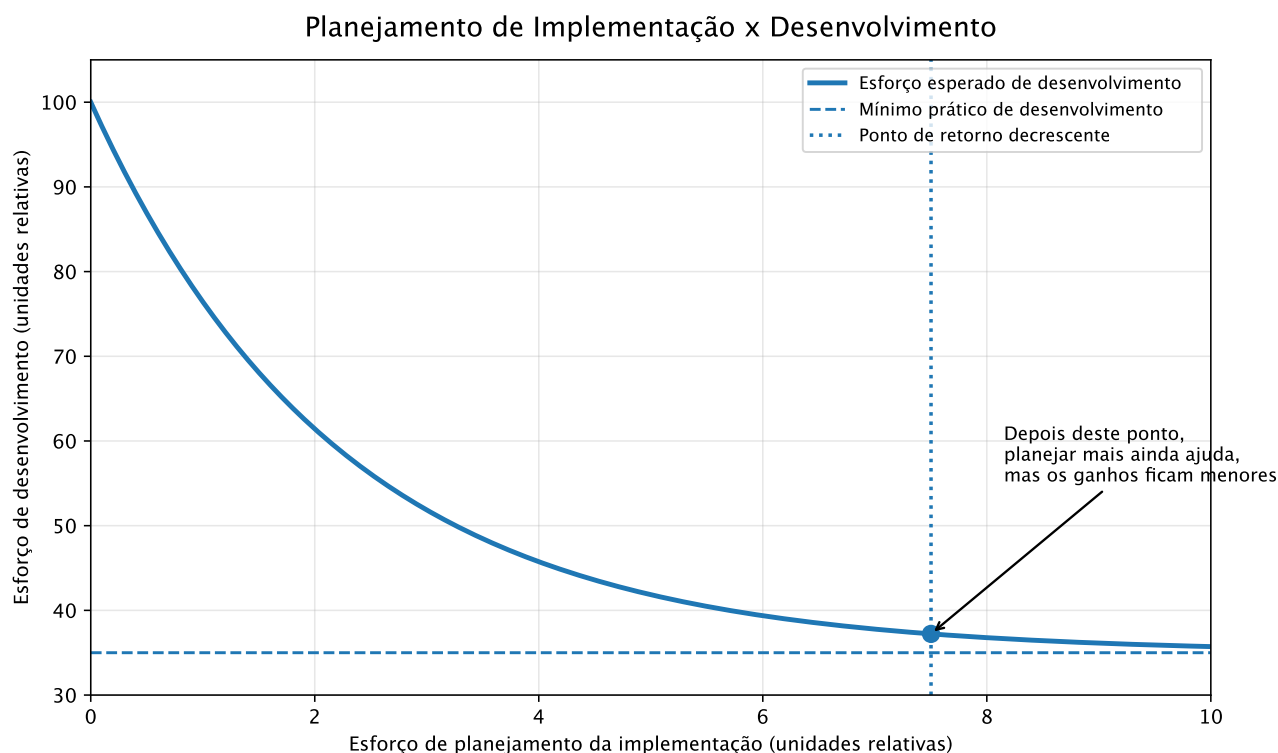
Quero que o plano responda a algumas perguntas práticas:

- O que vai mudar, o que está fora do escopo e quais sistemas, módulos, jobs, eventos ou APIs serão afetados? Quem é responsável pelos fluxos envolvidos?
- Quais regras de negócio e comportamentos legados precisam ser preservados? Quais padrões de código existentes devemos seguir ou evitar?
- Que estrutura e sequência de implementação fazem sentido, e quais premissas poderiam nos obrigar a mudá-las?
- Quais testes e casos de borda vão mostrar se a mudança funciona? O que podemos automatizar e o que ainda precisa de verificação humana?
- O que precisa acontecer em torno do release, incluindo scripts, migrações, backfills, feature flags, configuração e ordem de deploy? Quem cuida dos passos manuais, o que verificamos depois e como nos recuperamos se algo der errado?

As respostas podem caber em um ticket. Em uma mudança complicada, algumas talvez precisem de investigação própria. Importa mais saber se elas ajudam o próximo engenheiro a tomar uma decisão do que a quantidade de documentação produzida.

O que a curva explica bem

É assim que imagino a relação entre o planejamento útil e o esforço de desenvolvimento que vem depois:



Modelo ilustrativo — baseado em observação prática, não em análise formal de dados.

As unidades são ilustrativas. Este é um modelo mental baseado em experiência, não um estudo ou benchmark.

A queda acentuada no início representa o trabalho evitável que uma investigação curta pode revelar. Podemos descobrir que um fluxo já existe, que um campo tem um significado de negócio escondido ou que uma dependência se comporta de forma diferente em produção. Entender por que uma mudança parecida causou problemas pode alterar a abordagem antes que tenhamos muito código para desfazer. Uma hora procurando uma solução existente pode poupar horas implementando uma substituta.

É esse tipo de economia que tenho em mente: menos descobertas tardias, menos idas e vindas sobre quem é responsável por um fluxo e menos *"ah, isso também afeta aquele outro serviço"*.

Mas parte do trabalho só muda de momento. Ler um módulo durante o planejamento, em vez de durante o desenvolvimento, continua tomando tempo. Uma tarefa de desenvolvimento mais curta, por si só, não mostra que o time economizou esforço. A economia aparece quando esse entendimento antecipado evita trabalho que precisaríamos refazer ou nos ajuda a escolher uma implementação mais simples.

Em algum momento, a curva achata. Alguém ainda precisa escrever, testar, revisar, integrar e fazer deploy da mudança. Algumas perguntas também precisam de código funcionando para serem respondidas. Um documento mais longo pode nos deixar mais confiantes sem resolver essas perguntas.

O custo que falta no gráfico

Uma forma de esboçar a curva é a equação à qual sempre volto:

$$\text{Desenvolvimento}(P) = D_{\min} + (D_0 - D_{\min}) * e^{(-kP)}$$

Aqui, P é o esforço de planejamento e $\text{Desenvolvimento}(P)$ é o esforço esperado de desenvolvimento depois dele. D_0 é o esforço de desenvolvimento sem planejamento, enquanto $D_{\min}$ é o mínimo prático que permanece mesmo depois de um planejamento útil. O parâmetro k controla a velocidade com que a curva se aproxima desse mínimo; ele representa o quanto o planejamento reduz incerteza e retrabalho.

Com $D_0 > D_{\min}$ e $k > 0$, a curva sempre cai. Olhar apenas para ela faz parecer que mais planejamento é sempre melhor. Mas planejar também tem um custo.

Para um escopo fixo, com planejamento e desenvolvimento medidos nas mesmas unidades de esforço, a conta mais completa é $\text{Total}(P) = P + \text{Desenvolvimento}(P)$. Cada atividade entra em apenas um lado dessa soma: um spike exploratório contabilizado como planejamento não deve entrar também como desenvolvimento.

Nesse modelo simplificado, mais uma hora de planejamento só reduz o esforço total se economizar mais de uma hora depois. Quando a economia fica abaixo disso, o planejamento adicional aumenta o total, mesmo que a estimativa de desenvolvimento continue caindo. Em uma mudança que já entendemos bem, o planejamento adicional pode nem compensar o esforço gasto. Isso dá uma consequência prática ao achatamento da curva: precisamos de um motivo para continuar planejando.

Eu não usaria essa equação para estimar uma sprint ou calcular uma porcentagem de planejamento. Não tenho valores medidos para esses parâmetros. Ela pressupõe planejamento útil e escopo fixo, enquanto o planejamento real pode revelar trabalho que faltava e aumentar uma estimativa. Essa descoberta pode evitar um release incompleto; a estimativa menor teria sido enganosa.

O esforço também é só parte da decisão. Em uma mudança com consequências graves em caso de falha, eu dedicaria mais tempo a validar a recuperação, mesmo que isso não encurtasse a implementação. Um plano ainda precisa atender às necessidades de segurança e correção da mudança.

Escolha o próximo passo pelo que ele pode ensinar

Considere uma migração hipotética de um campo. Antes de alterar o schema, eu gostaria de saber quais jobs e serviços dependem desse campo, quais comportamentos legados precisam ser preservados e que ordem de deploy manteria esses consumidores funcionando. As respostas poderiam mudar a sequência de implementação.

Agora suponha que a dúvida restante seja como um backfill se comportará com dados representativos. Um teste limitado, em um ambiente seguro, pode ensinar mais do que outra discussão de design. O plano pode definir o que testar e qual resultado nos faria reconsiderar a abordagem. Então podemos executar o experimento e revisar o plano.

A mesma mudança pode precisar de mais investigação em uma área e de código funcionando em outra. Eu dedicaria mais esforço de planejamento a uma decisão difícil de reverter e usaria pequenos experimentos onde fosse possível aprender sem comprometer o restante do sistema.

É por isso que eu também revisitaria o plano durante o desenvolvimento. Quando a implementação expõe uma premissa errada, atualizar o plano ajuda a próxima tarefa a aproveitar o que aprendemos. Seguir a sequência original só porque a tarefa de planejamento está fechada perderia o propósito.

Como decido que já planejamos o suficiente

Já planejei de menos e de mais. A pressão para avançar rápido pode deixar perguntas básicas sobre impacto, testes ou rollout para interromper o desenvolvimento. O desejo de evitar erros pode nos manter buscando no documento respostas que precisam de um experimento.

Para mim, o ponto ideal continua em algum lugar entre caos e teatro. Quero que o time consiga dizer qual é a próxima mudança pequena e delimitada, explicar por que faz sentido começar por ela e descrever como vamos verificá-la. Os principais riscos e passos manuais do release devem estar visíveis, mesmo quando alguns detalhes ainda precisam de validação.

As incertezas restantes devem ser explícitas. Precisamos distinguir o que deve ser resolvido antes do release daquilo que podemos aprender com segurança durante a implementação, e combinar o que nos faria parar ou mudar de direção. Isso é mais útil do que perguntar se todos estão confiantes.

Antes de gastar mais tempo em um plano, quero perguntar: *"Que decisão o próximo passo de planejamento vai mudar, e por que planejar é a melhor forma de responder a essa pergunta?"*

Quando conseguimos apontar uma premissa que seria cara se estivesse errada e uma forma útil de verificá-la, vale continuar planejando. Quando a próxima resposta precisa de código funcionando, é hora de construir a menor parte capaz de nos dar essa resposta e levar o aprendizado de volta ao plano.